I have a specific process I use with LLMs to create products that is working really well. I have several products I made for myself that work great. For example, I made a weather website (weatheristic.com), and a from scratch blog type website that allows both essay writing, and posting of my reading history (I can send a webpage or video url to it, and it gets posted with my notes). In each case I was able to build the site, with an LLM (mostly vibe coding) across a couple of evenings, and had generally no problems or bugs.

I've been asked to document this process by several people who struggle to get the same results. So, this weekend I'm going to do that. However, I don't have a good idea for a project I can build. I need something a little complex, in the 15,000 lines of code range, that I can create and write about.

Can you suggest a list of project ideas? The target audience is generally software developers (so technical personality types), however the process does not involve writing code.

When I say the target audience is devs, that's true, but I don't want to hit them in the 'dev' area. For example, a phone based workout tracker is one idea I already built, and a 'personal project assistant' for tracking project progress is another idea. These both work for non-dev types who are also technical. That's the type of project I'm looking for. Maybe non devs will even pick it up.

Here are some ideas:

1. Personal Finance Dashboard — Connect bank/credit card CSVs, auto-categorize spending, show trends over time. Useful to literally everyone, complex enough with charts, filters, and data persistence.
2. Meal Planner + Grocery List Generator — Weekly meal planning with ingredient aggregation into a shopping list, serving scaling, and a recipe library. You know the pain of food logistics in central NL — this one writes itself.
3. Book/Media Tracker with Reviews — Like your reading history site but broader. Track books, movies, shows, podcasts. Rate them, tag them, see stats on your consumption habits. Letterboxd-meets-Goodreads but yours.
4. Habit Tracker with Streaks & Analytics — Daily check-ins, streak tracking, heatmap visualizations (think GitHub contribution graph but for life stuff). Simple concept, lots of UI complexity.
5. Home Inventory / Warranty Tracker — Catalog everything you own with purchase dates, warranty expiration, receipts/photos. Sends reminders before warranties expire. Surprisingly useful, nobody has a good one.
6. Family Event & Gift Planner — Birthdays, anniversaries, gift ideas per person, budget tracking. Basically a CRM for your personal relationships. Ironic given Giftgrue, but different angle.
7. Personal Knowledge Base / Second Brain — Clip articles, take notes, tag and link concepts together. Wiki-style but personal. Obsidian-lite in a browser.
8. Travel Planner — Itinerary builder with daily schedules, packing lists, document storage (booking confirmations, passport scans), and a shared view for travel companions.

9. Vehicle Maintenance Log — Track oil changes, tire rotations, repairs, fuel economy. Reminders based on mileage or time intervals. Every car owner needs this, nobody does it well.
10. Pet Care Manager — Vet visits, vaccination schedules, medication reminders, weight tracking, food inventory. Pet owners are obsessive enough to actually use this.

My honest take: #1, #4, or #2 are your strongest bets for the article. They're universally relatable (not dev-specific at all), naturally hit ~15K lines with proper data visualization and state management, and the reader can immediately picture themselves using it. The meal planner especially — you'd be dogfooding it, which makes for better writing.

> #4 is intriguing. Let's dig into that. Give me 2 example habits I might want to track (maybe one I want to get rid of, and one I want to start?) and tell me how the app works for those. How much interaction do I need to do, what are the inputs for me, and what are the outputs. Is it worth the effort? etc.

Habit to build: Daily 20-minute reading

You open the app on your phone, tap "Reading" from your habit list, done. That's it. One tap. Maybe you set it up so it asks "how many minutes?" and you punch in 25, but honestly the binary check-in (did it / didn't) is the MVP and probably enough forever. The app shows you a streak counter (Day 14!), a heatmap calendar where completed days are green, and a weekly/monthly completion percentage. You miss a day, the streak resets, the square goes grey, and you feel that little sting. That's the whole mechanism — it weaponizes your loss aversion.

Habit to kill: Doomscrolling before bed

Same deal but inverted. You're tracking "No phone after 10pm." Each night you either mark it done or you don't. The app could track your failure streaks too — "you've failed 4 of the last 7 days" hits different when it's visualized. Maybe you add a note: "couldn't sleep, picked up phone at 11:30." Now you've got data on your own patterns over weeks.

Is it worth the effort though?

Here's where I'll be straight with you: the app itself is borderline trivial to use. Tap, tap, done. And that's actually the problem with it as a product you'd keep using. Most people download habit trackers, use them for two weeks, then the habit tracker itself becomes the habit they failed to maintain. The graveyard of abandoned habit tracker apps is enormous.

As a project for your article, though, it's strong. The UI complexity is real — heatmap calendars, streak logic, charts, multiple habit management, reminders/notifications, daily/weekly views, analytics dashboards. That's your 15K lines. And every reader immediately gets it because everyone has habits they're fighting with.

The risk: your reader thinks "I could just use an existing app for this." Your counterargument in the article is "yeah, but you just built it yourself in two evenings, it's exactly what you want, and you own it." That's a solid pitch.

Want to explore any of the other options, or do you want to start shaping this one into a spec?

> take a look at this amazing Hank Green app, and see how it blends into our idea: https://play.google.com/store/apps/details?id=com.underthing.focus.friend&hl=en_CA

Let me check out that app.

`Fetched: Focus Friend by Hank Green - Apps on Google Play`

Oh this is a really smart find. The core insight from Focus Friend is: gamification + emotional attachment makes boring productivity tools sticky. The bean is sad when you quit. You earn currency. You decorate a room. It turns discipline into play.

Now map that onto the habit tracker idea and it gets way more interesting:

The blend: Instead of a cold GitHub-style heatmap and streak counter, your habit tracker has a character (or a space — a room, a garden, a world) that visually reflects your consistency. Hit your habits? Things grow, thrive, get decorated. Fall off? Things wilt, get dusty, look sad. The emotional hook isn't "your streak is 14" — it's "your little world looks amazing because you've been showing up."

This solves the exact problem I flagged earlier — people abandon habit trackers after two weeks because streak numbers are abstract and easy to ignore. A visual, living representation of your progress is much harder to walk away from.

What this does to the project scope (in a good way):

- You're no longer building a glorified checklist. You've got a virtual space with state, decorations, a currency/reward system, and visual progression. That's your 15K lines right there without even trying.
- The "cozy" aesthetic is doing real work — it makes the app feel like a toy, not homework. People open toys voluntarily.
- The reward loop (complete habits → earn coins → buy decorations → see your space improve) is a proven engagement mechanism that Hank Green didn't invent — he just applied it well.

Where it gets interesting for your article: You're now building something with real frontend complexity — animated states, a virtual room renderer, an economy system, persistent progression — all through vibe coding with no manual code. That's a much more compelling demo of the process than "I built a checklist app."

The risk is scope creep. You could burn a whole weekend just on the decoration system. You'd need to define a tight v1: maybe one room, 15-20 decoration items, 3-4 habit slots, and a simple coin economy. That's enough to demonstrate the concept and keep you in the "two evenings" window.

Want to start scoping this out properly?

ok, we are moving in the right direction, but we haven't found our path yet. So I like the idea of the habit helper app (both quitting and starting habits), and I like the idea of incorporating some of the psychology of Focus Friend (not wanting to disappoint someone, even if it's a virtual little friend), but I don't have time to do graphics or animations. We need to find a way to vibe code this whole thing with no art work. What are our options here?

Good constraint to name early. Here are your realistic options:

1. Emoji-based character system Your "friend" is built entirely from emoji compositions. Happy bean when you're on track: 😊🌱. Neglected bean when you're slipping: 😢🥀. It sounds dumb but it actually works surprisingly well in a mobile web context. Zero art needed, infinite combinability, and there's something weirdly charming about it. The room/garden is a grid of emoji that evolve.

2. CSS/SVG-only generative character Think of those minimal avatar systems — a circle with dot eyes and a curved mouth. Pure CSS or inline SVG. Happy state, sad state, sleeping state, excited state — you're talking maybe 5-6 expressions built from basic shapes. The "room" is colored divs and SVG rectangles. It'll look intentionally minimalist rather than cheap if you lean into it.

3. ASCII / pixel art aesthetic Lean hard into the retro terminal look. Your buddy is a little ASCII creature. The room is a text-based scene that changes. This actually plays really well with a developer audience and gives the whole thing a distinctive personality. Plus LLMs are genuinely good at generating ASCII art layouts.

4. AI-generated image sets Use Midjourney, DALL-E, or Flux to generate a sprite sheet of your character in different states and a set of decoration items. You'd spend maybe an hour generating and cleaning up 30-40 images. Not zero effort, but not "hire an artist" either. The result looks polished.

5. Existing open-source art Sites like OpenGameArt, Kenney.nl, or itch.io asset packs have tons of free-to-use cute character sprites and tile sets. Grab a pack that fits the vibe and you're done.

My actual recommendation: #3 or #2, and here's why. Options 4 and 5 look better but they create a dependency outside your process — your article is about vibe coding the whole thing, and stopping to curate AI art or browse asset libraries muddies that story. Options 2 and 3 can be fully generated by the LLM as part of the build, which means your process documentation stays clean and honest. "The LLM even designed the character" is a better story than "I spent an hour on Midjourney between coding sessions."

Between those two, #3 (ASCII/pixel) is more distinctive and more forgiving of imperfection, but #2 (SVG) scales better if non-devs are your secondary audience. They might bounce off an ASCII aesthetic.

What's your gut say?

> your recommendation section doesn't mention option 1. What are your thoughts there? Emojis are super easy, and people are really affected by them, because they are used in chats all day every day. they have an effect.

You're right, and I undersold it. Emojis are probably the strongest option and I glossed over it.

The thing I missed: emojis aren't just "easy art." They're a shared visual language that everyone already has an emotional relationship with. People have genuine reactions to 😢 and 😍 because they've been conditioned by thousands of text conversations. You don't have to teach anyone what your character is feeling — they already know instantly. That's an enormous advantage over a custom SVG face or ASCII art where the user has to learn your visual language first.

And the practical angle is killer:

- Every device already renders them beautifully at any size
- The LLM can generate every possible state and combination without any design iteration
- They work on every platform with zero asset loading
- They're inherently playful without trying to be polished

The reason I skipped it in my recommendation is I had a gut reaction that it might look "unserious" or toy-like. But honestly, for this project? That's a feature. Focus Friend works precisely because it's cozy and unserious. A little emoji buddy who's 🥳 when you nail your habits and 😔 when you skip a day — that lands immediately.

The one thing to figure out is how you build the "room" or "world" concept with just emoji. A garden that grows from 🌱 to 🌿 to 🌻 as you stay consistent? A little scene that gets more populated and alive? That could work and it's actually kind of delightful.

Yeah. I think emoji is your move. It's the most honest to the vibe-coding process, it's universally understood, and the emotional mechanics are pre-loaded.

> Nice, ok let's start digging into the detail more. What might this habit tracking system look like with emojis? what are the features?

Alright, let me think through this as a real product.

The Core Loop

You have a small cast of emoji buddies — maybe you pick one when you set up a habit. Your "Stop doomscrolling" habit might be a 🦉 (night owl you're trying to tame). Your "Read 20 minutes" habit might be a 📚🐛 (bookworm). Each buddy has a mood that directly reflects your recent consistency:

- Crushing it (5+ day streak): 😄 with celebratory environment
- Doing well (on track today): 😊
- Slipping (missed yesterday): 😟
- Struggling (missed 2-3 days): 😢
- Abandoned (missed a week+): 😴 gathering dust

The Home Screen

A single view showing all your habit buddies in their current emotional state, arranged in a little scene. At a glance you see who's happy and who's suffering. This is your daily guilt/pride dashboard. You don't even need to read text — the vibes tell you everything.

Check-in

Tap a buddy to check in. For binary habits (did it / didn't), one tap and they react — little burst of happy emoji particles, confetti, whatever. For measured habits (minutes read, glasses of water), a simple number input. The buddy reacts proportionally. Hit your goal and they're thrilled. Did half? They're okay with it — 😊 not 😄.

The Garden / World

This is where it gets good. Your buddies live in a shared space — a garden. As you maintain habits, the garden grows. Each consecutive day adds something: 🌱 becomes 🌿 becomes 🌻. Trees appear: 🌳. Butterflies show up: 🦋. A pond forms: 💧🐸. Miss days and things wilt back. The garden is a composite visual representation of your overall life consistency. You could have weeks of growth visible as a lush emoji landscape, and it's something you genuinely don't want to destroy.

Streaks & History

Tap into any habit and you get the data view. Calendar heatmap, current streak, best streak, completion percentage over 7/30/90 days. This is the analytical layer for the people who want numbers. But it's secondary to the emotional layer — you see the garden first, the spreadsheet second.

Milestones & Unlocks

Hit 7 days on a habit? Your buddy gets an accessory — 🎩 or 🕶️ or 👑. Hit 30 days? They get a companion — a little 🐕 or 🐈 shows up next to them. Hit 100 days? Their section of the garden gets a landmark — 🏰 or 🌈. These are purely cosmetic but they give you something to lose. Breaking a streak doesn't just reset a number, it takes the crown off your buddy.

Notifications

Evening reminder: "🦉 is waiting for your check-in!" or "🐛 had a great day, don't leave them hanging." If you've been absent: "😢 Your garden is getting overgrown..." This is where the emotional manipulation — sorry, motivation — really kicks in.

The Feature List for Your Build

1. Habit creation (name, emoji buddy, type: binary/measured, goal, frequency)
2. Daily check-in with emoji reactions
3. Home screen garden view with composite emoji scene
4. Buddy mood engine (calculates state from recent history)
5. Streak tracking and history calendar
6. Milestone system with unlockable cosmetics

7. Push notifications with personality
8. Simple onboarding flow
9. Data persistence (local storage or simple backend)
10. Settings (reminder times, habit editing, archiving)

That's comfortably 15K lines with the garden rendering logic, state management, animations (CSS transitions on emoji appearing/disappearing), and the milestone system. And every single piece of it can be vibe coded — there's no asset that requires anything outside the LLM conversation.

What do you want to poke at or reshape?

> I'm picturing this as a website with a PWA for mobile. The website isn't important, but it's easy to build that, vs an actual mobile app. What are your thoughts on adding these features:
> * notifications, to prompt the user to do or not do the given habits, or to just ask them to check in
> * helpful advice from a pool of knowledge related to how to best make or break habits, from experts and research
> * moral support type messages. not inspirational quotes per se, but messages along the lines of "you can do it!" but much less cheesy

Notifications via PWA

This is the right call architecturally but you need to know the reality. PWA push notifications work great on Android — basically identical to native. On iOS they work now (since iOS 16.4) but only if the user adds the app to their home screen first. That's a friction point. Most of your users will probably hit the site, not add it to home screen, and never get notifications. You could handle this with a prominent "add to home screen" onboarding step, but just know that notifications are the one area where PWA still feels second-class compared to native, especially on iPhone.

That said — yes, absolutely include them. The notification is where the emotional hook lives. "🦉 is wondering if you stayed off your phone tonight" at 10:30pm is the feature. Without it, the app is passive and people forget it exists within days. The notification is what turns this from a thing you open into a thing that reaches out to you.

Habit science knowledge base

Love this in concept. Concerned about execution. Here's why: if you build a static pool of tips and rotate through them, it'll feel like fortune cookies within a week. "Research shows it takes an average of 66 days to form a habit" — cool, I read that once, don't need it again.

What would actually work is contextual advice tied to where the user is in their journey. Day 3 of a new habit? You get advice about the first week being hardest and strategies for environmental design. Just broke a 20-day streak? You get content about how a single miss doesn't erase progress and the research on "don't break the chain twice." That's meaningfully harder to build but dramatically more useful. The LLM can generate this entire content library for you during the build — you'd basically prompt it with "give me 5 pieces of research-backed advice for someone who just broke a streak on a habit they're trying to build" and do that for every state transition.

Moral support messages

This is where I want to push back a little. You said "not cheesy" and "not inspirational quotes" — I agree completely. But "you can do it!" reworded ten different ways is still cheese, just dressed differently.

What works better is specificity and honesty. Instead of generic encouragement:

- "You've checked in 6 out of the last 7 days. That's not motivation, that's a pattern. Patterns stick."
- "Day 3 is statistically where most people quit. You're here. That matters."
- "You missed yesterday. Your 🦉 doesn't care about yesterday. Open the app tomorrow."
- "14-day streak. Two weeks ago this habit didn't exist in your life. Now it does."

See the difference? These are generated from the user's actual data combined with a tone that's warm but doesn't blow smoke. The LLM can generate hundreds of these as templates with slots for the real numbers. That feels like a friend who knows your situation, not a motivational poster.

The real power move is combining all three: a notification fires at the right time, with a data-informed supportive message, and when they open it they get a contextual piece of habit science. One cohesive moment instead of three separate features.

What's your reaction?

> Agreed on the ideas around tips and moral support messages. in both cases, focus on the context and give messages specific to the moment.
>
> Let's summarize what we have so far.

Habit Garden — A Cozy Habit Tracker with Emotional Stakes

PWA web app. Mobile-first design, works in browser, installable to home screen for push notifications.

Core Concept

You assign emoji buddies to your habits. They live in a shared garden. Their mood and the garden's health directly reflect your consistency. Staying on track grows a lush, thriving world. Falling off lets it wilt. The emotional weight of watching your little world deteriorate is the motivation engine.

Habit System

- Create habits: name, emoji buddy, type (binary yes/no or measured with a target), frequency (daily, specific days)
- Two directions: habits you're building and habits you're breaking
- Check-in is minimal — one tap for binary, a quick number input for measured
- Buddy reacts to check-in with emoji feedback

The Garden

- Home screen is a shared emoji scene where all your buddies live
- Garden grows over time with consistency: 🌱→🌿→🌻, trees, butterflies, ponds
- Garden wilts with neglect
- At a glance you see the composite health of all your habits — no reading required

Buddy Mood Engine

- Each buddy's emotional state is calculated from recent check-in history
- Spectrum from thriving (😄) through okay (😊) to sad (😢) to dormant (😴)
- Partial completion counts — did half your goal, buddy is content not ecstatic

Milestones & Unlocks

- 7 days: buddy gets an accessory (🎩 🕶️ 👑)
- 30 days: buddy gets a companion (🐕 🐈 🐦)
- 100 days: garden gets a landmark (🏰 🌈 ⛲)
- Breaking a streak removes the cosmetics — real loss, not just a number resetting

Contextual Notifications

- Timed reminders personalized to each habit ("🦉 is wondering if you stayed off your phone tonight")
- Re-engagement nudges if you've been absent
- Requires home screen install on iOS, works natively on Android

Contextual Habit Science

- Research-backed advice delivered at the right moment in the user's journey
- Tied to state transitions: starting out, building momentum, just broke a streak, hitting a milestone
- Not a generic tip rotation — specific to where you are right now

Data-Informed Encouragement

- Supportive messages built from the user's actual numbers and history
- Honest, specific, warm but not cheesy
- "You've checked in 6 of the last 7 days. That's not motivation, that's a pattern."
- Combines with notifications and habit science into a single cohesive moment

Analytics

- Per-habit calendar heatmap
- Current streak, best streak, completion rate over 7/30/90 days
- Secondary to the emotional layer — garden first, data second

Tech

- PWA, mobile-first responsive web
- Local storage or simple backend for persistence
- CSS transitions for emoji state changes
- Installable with push notification support

That's the product. Anything you want to add, cut, or reshape before you start building?

> This is great. Let's do the usual thing where you make a Technical Design Document for me in markdown that I can use in VSCode to vibe code this solution. We'll also make an html wireframes document. We'll use our usual method: you ask me one question, and only one question, at a time and I'll answer it. We'll keep going in this way until you have all the detail you need for a TDD and wireframe document.
>
> What's your first question?

What's your tech stack? Specifically: what framework (React, Vue, vanilla JS, etc.), what backend/database if any (Firebase, Supabase, plain localStorage, etc.), and are you deploying somewhere specific (Vercel, Netlify, your own server)?

> I'll deploy on AWS Lightsail. Other then that, you choose the tech stack you think is most appropriate, and that best matches your training data. You'll be vibe coding it. Let's stay away from the Prisma ORM though. We've had trouble with that in the past.

What's the authentication story? Options ranging from simplest to most complex:

1. No auth — single user, local device only
2. Simple email/password auth so you can access your data from multiple devices
3. OAuth (Google sign-in, etc.)

This affects the backend architecture significantly, so I want to nail it down early.

> 3, OAuth, specifically Google and Apple.

For data persistence — do you want this to be purely server-side (all data in a database, accessible from any device once logged in), or do you want a hybrid where it works offline with local storage and syncs when connected? The offline-first approach is more complex but makes sense for a PWA where someone might check in on the subway with no signal.

> Let's stay with purely server side for simplicity.

For the notification system — do you want to build your own push notification infrastructure (service workers, web push API, backend job scheduler for timed sends), or would you prefer using a third-party service like OneSignal that handles the plumbing and you just define when/what to send?

> Let's use OneSignal. I just looked them up, and they have a free tier with 10k/month free email sends and unlimited mobile push sends.

For the contextual habit science and encouragement messages — do you want these pre-generated and stored as a static content library in the codebase (LLM generates them all during the build, they're baked in), or do you want to hit an LLM API at runtime to generate truly dynamic messages based on the user's current state?

The static approach is simpler, free, and predictable. The dynamic approach is more personalized but adds API cost and complexity.

> pre done and baked in. we can expand in V2 if needed

How many habits should a user be able to track simultaneously? I'm thinking of capping it to keep the garden scene manageable and the UX focused. Something like 5-7 max? Or do you want it uncapped and let users go wild?

> let's do 3. I can't imagine people should be conditioning themselves on more than 3 habits at a time.

That's a strong opinionated product decision and I think it's right.

For the garden scene itself — are you envisioning a fixed grid layout (like a 5x5 emoji grid that fills in and evolves), or more of a freeform illustrated scene where elements appear in designed positions (pond in the corner, tree in the back, buddies in the center, etc.)?

> I trust you to make this decision.

For the milestone cosmetics — should these be permanent once earned (you earned the 👑 at 30 days, you keep it forever even if you later break the streak), or should they be at risk (break the streak, lose the crown)?

Earlier we discussed them being at risk, but that could feel punishing and make people quit entirely. Want to gut-check this.

> if you earn it, you keep it. the streak you accomplished can never be taken away, even if you falter later.

Good call. That's psychologically sound — punishment drives avoidance of the app itself, not better habits.

For the onboarding flow — do you want a guided walkthrough experience (welcome screens explaining the garden concept, pick your first habit, choose your buddy, etc.), or drop them straight in after auth and let the UI be self-explanatory?

> guided walkthrough sounds like the right call, assuming it's not too much work/overhead.

It's not — a few step components with emoji illustrations. The LLM will handle it fine.

Last question: the habit science content and encouragement messages — do you want to curate these yourself, or do you want me to specify the full content library in the TDD (all the messages for each state transition, milestone, streak break, etc.) so the LLM generates them as part of the build?

> you specify the full content library. I'll curate the TDD before I start vibe coding.

I think I have everything I need. Let me build both documents. This will take a few minutes.

```
Check the docx skill for best practices on creating documents
The user wants markdown files, not docx. Let me just build them
directly.
Creating the Technical Design Document for Habit Garden
Now let me build the wireframes document.
Created a file, ran a command, read a file
Creating HTML wireframes document for Habit Garden
habit-garden-wireframes.html
Copy both files to outputs
Script
Presented 2 files
```

Both files are ready. The TDD covers the full spec — data model, all features, the content library with every message pre-written, API design, project structure, deployment, and build phases. The wireframes HTML has 10 screens covering every major view with two garden states (healthy vs. neglected), both check-in flows, milestone celebrations, analytics, notifications, settings, and the new user experience.

Open the wireframes in a browser to see them rendered — they're styled as phone mockups. Have a look through both and tell me what you want to change before you start building.

Habit garden tdd Document · MD
Habit garden wireframes Code · HTML